

Preguntas de promoción FOD - con justificaciones

Se asume que las respuestas **en negrita** son las correctas.

Fuentes

A. Clases teóricas del 2024:

Clase 1. Conceptos básicos de BD, archivos.

Clase 2. Algoritmia básica y clásica.

Clase 3. Viaje del byte, tipos de archivos, claves, eliminación.

Clase 4. -

Clase 5. Índices.

Clase 6. Partes 1, 2, 3 y 4: árboles, árboles balanceados.

Clase 7. Partes 1 y 2: hashing, propiedades, dispersión.

B. Libro de la bibliografía “Introducción a las bases de datos: fundamentos y diseño” por Bertone, Rodolfo Alfredo y Thomas, Pablo Javier. ISBN 9789876151368. Se utilizó la parte correspondiente a esta materia, que figura como “LIBRO SOLO FOD”.

Puede vs. Debe

En el contexto de la materia, la distinción entre "se puede" y "se debe" es fundamental para comprender la naturaleza de las operaciones y las decisiones de diseño en la gestión de la información. Estos términos reflejan grados de **posibilidad, obligación o resultado inherente** a una acción o característica del sistema:

"Se Puede" (Posibilidad / Alternativa)

Este término indica que una acción o situación **es factible o una de varias opciones posibles**, no necesariamente una obligación o un resultado garantizado en todas las circunstancias.

Ejemplos:

- **En la gestión de espacio de archivos:** Tras una baja lógica, **se puede reaprovechar** el espacio disponible con nuevas altas, utilizando técnicas como la "reasignación de espacio" [LIBRO SOLO FOD, p. 63, 174]. Esto es una **alternativa posible** para reutilizar los lugares marcados como borrados, insertando nuevos registros allí [LIBRO SOLO FOD, p. 174] [Clase 3, p. 34]. Sin embargo, si no hay espacio disponible o apto, el registro simplemente se agrega al final del archivo [LIBRO SOLO FOD, p. 178] [Clase 3, p. 34].
- **En archivos con registros de longitud variable:** Para insertar un elemento, **se puede** localizar el lugar más adecuado según estrategias como "primer ajuste", "mejor ajuste" o "peor ajuste" [LIBRO SOLO FOD, p. 192] [Clase 3, p. 40]. Esto resalta que es una elección de política de colocación, no una imposición universal.

- **En la organización de archivos:** Un archivo **puede** contener un tipo de dato simple o una estructura heterogénea como registros [LIBRO SOLO FOD, p. 67].
- **En el uso de índices:** Con el uso de una fuente de información organizada adicional (un índice), **se puede lograr mejorar** la performance de acceso a la información.
- **En hashing:** Hashing **puede** asegurar un acceso para recuperar un dato en promedio, no en todos los casos [LIBRO SOLO FOD, p. 155] [Clase 7 parte 2, p. 6]. Si bien "no es posible asegurar" que cualquier registro se encuentre en un solo acceso, la gran mayoría de las búsquedas sí lo serán [LIBRO SOLO FOD, p. 155].

"Se Debe" (Obligación / Requisito / Restricción)

Este término denota una **condición obligatoria, un requisito fundamental o una restricción ineludible** para el correcto funcionamiento o la coherencia del sistema o algoritmo [LIBRO SOLO FOD, p. 88, 199].

- **En operaciones básicas de archivos:** Para operar con un archivo desde un algoritmo, **se debe realizar la apertura** [LIBRO SOLO FOD, p. 77]. Una vez finalizada la operatoria sobre un archivo, el mismo **debe ser cerrado** [LIBRO SOLO FOD, p. 81]. Antes de realizar una operación de lectura, un algoritmo **debe controlar el fin de archivo** (EOF) [LIBRO SOLO FOD, p. 93].
- **En la definición de datos:** La variable utilizada para lectura o escritura en un archivo **debe ser del mismo tipo** que los elementos declarados para el archivo [LIBRO SOLO FOD, p. 83].
- **En la gestión de espacio con registros de longitud variable:** Cuando se desea recuperar el espacio borrado lógicamente con nuevos elementos, **deben tenerse en cuenta nuevas consideraciones**, como que el nuevo elemento **debe "caber"** en el espacio disponible [LIBRO SOLO FOD, p. 191] [Clase 3, p. 37]. Además, el programador **debe resolver en forma mucho más minuciosa** las operaciones de agregar y quitar elementos [LIBRO SOLO FOD, p. 187].
- **En los árboles B:** Para poder borrar un elemento de un árbol B, el mismo **debe estar localizado en un nodo terminal** (o ser intercambiado con uno) [LIBRO SOLO FOD, p. 312] [Clase 6, p. 20]. La altura de un árbol B **debe ser la misma** desde la raíz hasta cada nodo terminal [LIBRO SOLO FOD, p. 291] [Clase 6 parte 1, p. 25].
- **En hashing:** La función de hash **debe esparcir** los registros de la manera más uniforme posible [LIBRO SOLO FOD, p. 372]. Además, **se debe contar con algún método** para reubicar los registros que no pueden ser almacenados en la dirección base (overflow) [LIBRO SOLO FOD, p. 387]. Cuando el archivo se completa, **es necesario obtener mayor cantidad de direcciones** para los nodos [LIBRO SOLO FOD, p. 415]. Hashing **no puede** usarse con registros de longitud variable, ni para mantener un orden físico de los datos, ni con claves duplicadas [LIBRO SOLO FOD, p. 359, 360] [Clase 7 parte 2, p. 7].

Multiple Choices

Archivos

1. Los archivos con registros de longitud variable
 - a. Ocupan menos espacio que los archivos con registros con longitud fija
 - b. Ocupan más espacio que los archivos con registros con longitud fija
 - c. Ocupan el mismo espacio que los archivos con registros con longitud fija
 - d. Ninguna de las anteriores**

Esto se debe a que, si bien el objetivo principal de los registros de longitud variable es optimizar el uso del espacio al utilizar solo los bytes necesarios y evitar el desperdicio común en registros de longitud fija, su implementación requiere información adicional. Esto significa que no siempre, en todas las circunstancias y de forma estricta, ocuparán menos espacio, invalidando la opción 'a' y, por ende, las demás opciones que proponen igualdad o mayor ocupación de espacio son incorrectas para su propósito general. Además, "ocupan" implica obligación para cada sentencia y, volviendo a lo anterior, no siempre ocuparán menos, más o el mismo espacio.

2. Un algoritmo de búsqueda en un archivo:
 - a. Es más eficiente si el archivo está ordenado
 - b. Puede ser más eficiente si se considera como precondition que el archivo está ordenado.**
 - c. Es igual de eficiente si el archivo está ordenado o desordenado.
 - d. Ninguna de las anteriores

El ordenamiento de un archivo actúa como una precondition que posibilita el uso de algoritmos de búsqueda mucho más eficientes. Mientras que una búsqueda en un archivo desordenado (serie) es de orden lineal ($O(N)$), tener el archivo ordenado permite aplicar, por ejemplo, la búsqueda binaria, la cual reduce drásticamente la eficiencia a un orden logarítmico ($O(\log_2(N))$). Además, las estructuras de índices (como los árboles B o B+) también se benefician del orden, ofreciendo accesos más rápidos y logarítmicos. El término "puede ser" es crucial, ya que el ordenamiento no garantiza la eficiencia por sí mismo, sino que habilita métodos que sí la ofrecen

3. El proceso de merge de archivos
 - a. Requiere que todos los archivos estén ordenados
 - b. Requiere que todos los archivos estén ordenados por el mismo criterio
 - c. Puede realizarse con los archivos ordenados**
 - d. No puede realizarse sin los archivos ordenados
 - e. Algunas de las anteriores
 - f. Ninguna de las anteriores

Mientras que algunos algoritmos y propósitos de fusión se benefician enormemente o incluso requieren que los archivos estén ordenados, la afirmación de que el proceso de merge en general

requiere que todos los archivos estén ordenados es falsa. Hay muchas operaciones de fusión que pueden realizarse sin este pre-requisito.

4. El proceso de alta de registro por ajuste óptimo
 - a. Se puede realizar con registros de longitud fija
 - b. Se debe realizar con registros de longitud fija
 - c. Se puede realizar con registros de longitud variable
 - d. Se debe realizar con registros de longitud variable
 - e. Ninguna de las anteriores**
-

5. La operación assign
 - a. Se usa para abrir un archivo
 - b. Se usa para posicionarse en el primer registro del archivo
 - c. Vincula el archivo lógico con el archivo físico**
 - d. Se usa para saber la longitud del archivo
 - e. Alguna de las anteriores

Asigna un nombre a F, que puede ser de cualquier tipo de archivo. Esta llamada no abre el archivo, sino que simplemente asigna un nombre a una variable de archivo y lo marca como cerrado. ¹ Reset y seek posicionan el puntero del archivo al inicio.

6. Dado el NRR
 - a. Se puede acceder a un registro de un archivo fragmentado en sólo un acceso
 - b. Se puede acceder a un registro de un archivo no fragmentado en sólo un acceso
 - c. Se puede acceder a un registro de un archivo en sólo un acceso
 - d. Se puede lograr acceso directo a un archivo
 - e. Todas las anteriores**
 - f. Algunas de las anteriores.

El NRR proporciona la capacidad de direccionar de forma única y directa una posición específica en un archivo de registros de longitud fija, garantizando un acceso en una sola operación a disco, sin importar si hay espacio desperdiciado dentro de los registros (fragmentación interna) o si el archivo tiene registros marcados lógicamente como borrados (que aún ocupan su espacio físico fijo).

7. Un archivo fragmentado
 - a. Debe compactarse para optimizar el espacio utilizado
 - b. No debe compactarse para optimizar el espacio utilizado
 - c. A veces puede compactarse**
 - d. Nunca debe compactarse
 - e. Algunas de las anteriores

¹ <https://www.freepascal.org/docs-html/rtl/system/assign.html>

La fragmentación interna (espacio desperdiciado dentro de un registro) no es compactable de la misma manera que la fragmentación externa (espacios vacíos entre registros) o el espacio dejado por bajas lógicas, los cuales sí pueden recuperarse y compactarse. Además, incluso cuando la compactación es posible, a menudo implica un alto costo en términos de rendimiento, como leer y reescribir grandes volúmenes de datos. ²

- Fragmentación Interna:
 - Se produce cuando a un elemento de dato se le asigna mayor espacio del necesario.
 - Es común en registros de longitud fija, donde un campo tipo string podría almacenar un nombre corto como "Juan Pérez" y desperdiciar los 40 caracteres restantes dentro de su espacio asignado.
 - Esta fragmentación no puede compactarse mediante un proceso de reubicación de registros para unir espacios vacíos, ya que el espacio desperdiciado está dentro del propio registro y es parte de su asignación de longitud fija.
- Fragmentación Externa:
 - Se refiere al espacio disponible entre dos registros que es demasiado pequeño para ser reutilizado.
 - Este tipo de fragmentación puede surgir cuando se utilizan registros de longitud variable y se asigna solo el espacio estrictamente necesario (como en la técnica de "peor ajuste" para reasignar espacio).
 - El espacio resultante de las bajas lógicas (registros marcados como borrados pero que siguen ocupando espacio físico) también puede llevar a fragmentación, ya sea interna o externa, dependiendo de la estrategia de reasignación de espacio.
 - La fragmentación externa sí puede compactarse. Los "espacios no asignados" de la fragmentación externa o el espacio liberado por bajas lógicas pueden ser recuperados periódicamente mediante un proceso de compactación, a veces denominado "garbage collector" o mediante procesos de baja física que generan un nuevo archivo o reacomodan los existentes.

-
8. Un algoritmo de actualización maestro-detalles
 - a. Requiere que todos los archivos tengan la misma estructura
 - b. Puede realizarse entre archivos con diferente estructura**
 - c. Requiere que los archivos estén ordenados
 - d. Requiere que los archivos estén desordenados
 - e. Algunas de las anteriores

Siempre y cuando se puedan leer los datos necesarios del archivo detalle y del archivo maestro, y se puedan escribir los datos actualizados en el maestro, la diferencia en la estructura física subyacente (texto, fijo, variable) no impide la ejecución del algoritmo de actualización.

Razones:

- Visión Lógica vs. Física del Archivo: Los algoritmos, desde el punto de vista práctico, operan con una visión lógica de los archivos, utilizando nombres lógicos que luego se

² Clase teórica de FOD "Bajas"

"conectan" con los archivos físicos en el disco mediante la operación assign. Esto significa que el programa se enfoca en el contenido de los datos (qué datos son, cómo se relacionan lógicamente) más que en su almacenamiento físico exacto.

- Operaciones Genéricas de Lectura y Escritura: Las operaciones read y write permiten transferir información entre el archivo y una variable en el algoritmo (var_dato). El sistema operativo, a través de buffers de memoria, se encarga de optimizar esta transferencia, independientemente de la estructura interna del archivo. Mientras que var_dato debe coincidir con el tipo de datos de los elementos del archivo, el programa puede definir y procesar tipos de datos variados, desde simples como enteros o caracteres hasta estructuras heterogéneas como registros.
- Manejo de Diferentes Estructuras de Registros:
 - Los archivos pueden contener registros de longitud fija (tipados), donde el tamaño de cada elemento está predeterminado por la definición de sus campos.
 - También pueden manejar registros de longitud variable, donde el espacio utilizado por cada elemento no se fija de antemano y requiere que el programador gestione delimitadores de campo o indicadores de longitud.³
 - Además, existen los archivos de texto, que son caracteres estructurados en líneas y tienen un acceso exclusivamente secuencial, siendo útiles para importar y exportar datos.
- Interoperabilidad Demostrada: Un ejemplo clave se encuentra en los procesos de carga y exportación de datos. Las fuentes muestran cómo se puede leer desde un archivo de texto (Text) y escribir en un archivo binario tipado (File of tRegistroVotos). Esto ilustra que es posible extraer datos lógicos (como codProv, codLoc, nroMesa, etc.) de un archivo de texto y organizarlos en una estructura de registro para ser escritos en un archivo binario.⁴
- Enfoque en el Contenido Lógico para la Actualización: En un proceso de actualización maestro-detalle, se busca "modificar el contenido de un archivo [maestro] por el contenido de otro archivo [detalle]". Los algoritmos buscan y comparan claves (como regm.cod y regd.cod), y actualizan campos específicos (regm.stock). Estas operaciones se centran en el valor lógico de los datos, y el sistema de archivos subyacente gestiona cómo esos valores son almacenados y recuperados en cada tipo de archivo. La precondition de que "Ambos archivos están ordenados por igual criterio" se refiere al orden lógico de las claves, no a su estructura física.⁵

-
9. Un archivo serie
- a. Está ordenado
 - b. Puede ordenarse**
 - c. Requiere ordenarse
 - d. No requiere ordenarse
 - e. No está ordenado
 - f. Algunas de las anteriores

³ Clase teórica de FOD "Archivos"

⁴ Clase teórica de FOD "Archivos_texto"

⁵ Clase teórica de FOD "Maestro - Detalle"

Los archivos serie, por definición, son aquellos que no tienen ningún orden preestablecido más que el físico. Esto significa que los registros se almacenan en el orden en que fueron ingresados. Sin embargo, estos archivos pueden ser ordenados para mejorar la performance en la búsqueda de datos.

10. Dado un archivo con 1000 registros

- a. Siempre se puede llevar a memoria RAM para hacer búsquedas eficientes
- b. No siempre se puede llevar a memoria RAM para hacer búsquedas eficientes**
- c. Siempre puede realizarse búsqueda dicotómica
- d. No puede realizarse búsqueda dicotómica
- e. Algunas de las anteriores

Aunque 1000 registros puedan parecer pocos, la viabilidad de cargarlos completamente en RAM depende directamente del tamaño de cada registro. La memoria RAM tiene una capacidad de almacenamiento más limitada en comparación con los dispositivos de almacenamiento secundario, como los discos rígidos.

Cargar un archivo completo en la memoria principal (RAM) para ordenarlo o buscar es la "mejor alternativa en cuanto a performance" porque las operaciones en RAM son del orden de nanosegundos, mientras que en disco son del orden de milisegundos, una diferencia sustancial de velocidad. Sin embargo, esta estrategia "solamente puede ser utilizada para archivos pequeños". Si el tamaño total del archivo (número de registros multiplicado por el tamaño de cada registro) excede la RAM disponible, simplemente no cabrá en ella.

Si el archivo es "realmente grande" y no cabe en la memoria RAM, incluso transferir solo las claves y los Números Relativos de Registro (NRR) a la RAM puede ser inviable, requiriendo estrategias de ordenamiento externo como el "Merge Sort". Por lo tanto, aunque es deseable para la eficiencia, la capacidad física de la RAM es la limitante fundamental.

11. Un archivo ordenado

- a. Puede desordenarse**
- b. Conviene mantenerlo ordenado
- c. No conviene mantenerlo ordenado
- d. No puede desordenarse
- e. Algunas de las anteriores
- f. Ninguna de las anteriores

Un archivo ordenado puede desordenarse si las operaciones de manipulación de datos (inserción, eliminación, modificación) no incluyen mecanismos para mantener activamente ese orden, o si se opta por estructuras que priorizan la velocidad de la operación individual sobre el mantenimiento constante del orden físico, como en el caso de un archivo de datos que es un "archivo serie" con un índice separado que impone el orden lógico.

12.Cuál de las siguientes definiciones corresponde a archivo:

- a. Colección de registros que abarca un conjunto de entidades con ciertos aspectos en común y organizados para un propósito particular.

- b. Colección de registros semejantes almacenados en disco rígido.
- c. Colección de registros del mismo tiempo almacenados en un dispositivo de memoria secundaria.

d. Todas las opciones son correctas

- a. *Colección de registros que abarca un conjunto de entidades con ciertos aspectos en común y organizados para un propósito particular.* Esta definición es literalmente extraída de una de las definiciones de archivo en las fuentes: "Un archivo es una colección de registros que abarcan entidades con un aspecto común y originadas para algún propósito particular." (Bertone & Thomas, 2011)
- b. *Colección de registros semejantes almacenados en disco rígido.* Las fuentes definen un archivo como una "colección de registros semejantes". Además, especifican que los archivos deben residir en "dispositivos de almacenamiento permanente de información", dando como "ejemplo más natural" el "disco rígido", y contrastándolos con la memoria RAM que es volátil.
- c. *Colección de registros del mismo tipo almacenados en un dispositivo de memoria secundaria.* Esta definición se apoya en que un archivo es una "estructura de datos que recopila, en un dispositivo de almacenamiento secundario de una computadora, una colección de elementos del mismo tipo". También se reitera que los archivos residen en "dispositivos de memoria secundaria o almacenamiento secundario". (Asumo que "del mismo tiempo" en la pregunta es un error tipográfico y se refiere a "del mismo tipo", que es el concepto clave en la homogeneidad de los archivos según las fuentes).

13. Un archivo que maneja registros de longitud fija necesita

- a. Delimitadores que indiquen el fin de cada campo
- b. Delimitadores que indiquen el fin de cada registro
- c. Indicadores de longitud de registros

d. Ninguna de las anteriores

Un archivo que maneja registros de longitud fija no necesita delimitadores ni indicadores de longitud por las siguientes razones, según las fuentes:

- Longitud Predeterminada y Constante: En archivos con registros de longitud fija, la longitud de cada registro es predecible y está determinada por la suma de las longitudes de sus campos componentes. Esto significa que todos los registros ocupan el mismo tamaño de espacio.
- Responsabilidad del Sistema Operativo: Una ventaja crucial de los registros de longitud fija es que el proceso de entrada y salida de información (desde y hacia los buffers) es responsabilidad del sistema operativo. El sistema ya conoce la cantidad de bytes que conforman cada registro y puede gestionarlos de manera eficiente, transfiriendo bloques completos de datos sin necesidad de marcas internas.
- Delimitadores e Indicadores para Longitud Variable: Los delimitadores de fin de campo o de registro y los indicadores de longitud de campo y/o registro son mecanismos que se utilizan específicamente para manejar registros de longitud variable. En estos casos, el tamaño de cada elemento no se conoce de antemano, por lo que estas marcas son indispensables para saber dónde comienza y termina cada campo o registro.

-
14. El proceso de baja en un archivo con registros de longitud variable:
- a. Puede recuperar el espacio disponible con nuevas altas
 - b. Puede recuperar el espacio disponible compactando periódicamente el archivo
 - c. Puede recuperar el espacio disponible compactando el archivo ante cada baja
 - d. Todas las anteriores**

Todas las opciones son correctas porque cada una describe una estrategia válida para gestionar el espacio tras una baja en archivos con registros de longitud variable:

- a. *Puede recuperar el espacio disponible con nuevas altas*: Las fuentes explican la "Reasignación de espacio" como la reutilización de lugares marcados como borrados para insertar nuevos registros [LIBRO SOLO FOD, p. 63], especialmente relevante para registros de longitud variable con estrategias de "primer ajuste", "mejor ajuste" o "peor ajuste" [Clase 3, p. 37, 40].
- b. *Puede recuperar el espacio disponible compactando periódicamente el archivo*: Esto corresponde a la "Recuperación de espacio" que implica "periódicamente utilizar el proceso de baja física para realizar un proceso de compactación del archivo" [LIBRO SOLO FOD, p. 63]. Este método es aplicable a registros de longitud variable, ya que la baja es "similar a lo discutido anteriormente".
- c. *Puede recuperar el espacio disponible compactando el archivo ante cada baja*: Esta es una forma de "Baja física" que busca "recuperar el espacio físico" [LIBRO SOLO FOD, p. 56, 166] inmediatamente. Una técnica para esto es "Utilizar el mismo archivo de datos, generando los reacomodamientos que sean necesarios" [LIBRO SOLO FOD, p. 58, 172], con el objetivo de que el archivo "ocupe el lugar mínimo necesario" [LIBRO SOLO FOD, p. 167] en todo momento.

-
15. En el caso de realizar un alta de un registro
- a. se reaprovecha algún espacio de registro dado de baja previamente
 - b. se puede reaprovechar algún espacio de registro dado de baja previamente**
 - c. se debe reaprovechar si los registros son de longitud fija
 - d. se debe reaprovechar si los registros son de longitud variable
 - e. algunas de las anteriores son correctas
 - f. ninguna de las anteriores son correctas

a. *se reaprovecha algún espacio de registro dado de baja previamente*: La formulación "se reaprovecha" implica que esto siempre ocurre o es la acción por defecto, lo cual no es cierto. Depende de la política de gestión del archivo y de si existe o no espacio disponible apto para ser reutilizado [LIBRO SOLO FOD, p. 63].

c. *se debe reaprovechar si los registros son de longitud fija*: El "se debe" impone una obligación. Aunque para registros de longitud fija cualquier espacio liberado es teóricamente apto para una nueva alta (ya que todos los registros tienen el mismo tamaño) [Clase 3, p. 35], el sistema puede optar por no reutilizarlo (por ejemplo, si se usa solo baja lógica sin reasignación posterior, o si se agrega siempre al final) [LIBRO SOLO FOD, p. 73]. La reutilización es una ventaja o una técnica, no un requerimiento estricto para cada operación de alta [Clase 3, p. 34].

d. *se debe reaprovechar si los registros son de longitud variable*: Similar a la opción anterior, el "se debe" es restrictivo. Para registros de longitud variable, la reutilización del espacio es aún más compleja, ya que el nuevo registro debe "caber" en el espacio liberado [Clase 3, p. 37]. Se discuten estrategias como "primer ajuste", "mejor ajuste" o "peor ajuste" para encontrar un lugar, lo que subraya que es una política de colocación, no una obligación universal ante cada alta [Clase 3, p. 40].

e. *algunas de las anteriores son correctas*: Si solo la opción 'b' es la correcta en su formulación, esta opción sería engañosa ya que implica que "más de una" de las opciones (a, b, c, d) son correctas.

f. *ninguna de las anteriores son correctas*: Esta opción sería válida si ninguna de las anteriores fuera precisa. Sin embargo, la opción 'b' sí es correcta.

En resumen, la clave está en el matiz de "**se puede**" (posibilidad) frente a "se reaprovecha" (acción automática/general) o "**se debe**" (obligación), lo cual se alinea perfectamente con la descripción de las técnicas de recuperación y reasignación de espacio en las fuentes.

16. Un índice secundario tiene eficiencia de búsqueda

- a. lineal
- b. logarítmica
- c. constante y es 1
- d. constante y puede tender a 1
- e. Algunas de las anteriores son válidas
- f. ninguna de las anteriores son válidas**

Ninguna de las opciones es válida ya que no se especifica en dónde está organizado el índice secundario (árbol B, B+, B*, tabla hash, etc.).

Árboles

17. Un árbol avl

- a. tiene eficiencia de búsqueda logarítmica**
- b. puede tener eficiencia de búsqueda logarítmica
- c. puede tener eficiencia de búsqueda lineal
- d. Las opciones a y b son correctas
- e. Las opciones b y c son correctas
- f. Las opciones a y c son correctas
- g. Ninguna opción es correcta

Un árbol AVL tiene eficiencia de búsqueda logarítmica [6, 147; FOD - CLASE 6, 17] debido a su propiedad de balanceo [6, 146; FOD - CLASE 6, 15].

- Fundamento de la búsqueda logarítmica en árboles binarios: En un árbol binario de búsqueda, si está balanceado, la información se busca comenzando desde el nodo raíz y explorando hacia los nodos hoja [5, 98; 6, 129]. En cada paso, se compara el elemento buscado con el elemento en el nodo actual, y se decide si la búsqueda continúa por la izquierda (si es menor) o por la derecha (si es mayor), descartando así la mitad de los

elementos restantes [5, 98; 6, 129; 6, 136]. Este proceso de reducción del espacio de búsqueda a la mitad en cada paso es lo que le otorga una eficiencia de orden logarítmico ($\log_2(N)$) [5, 98; 6, 129; 6, 136; FOD - CLASE 6, 17].

- Problema con los árboles binarios no balanceados: La eficiencia logarítmica de un árbol binario solo se mantiene si el árbol está balanceado. Si un árbol binario se "desbalancea fácilmente" [FOD - CLASE 6, 15] o se "degenerado" y se transforma en una estructura tipo lista, su rendimiento de búsqueda decae a orden lineal [6, 145; FOD - CLASE 6, 15], es decir, deja de ser logarítmico.
- Rol del árbol AVL: Los árboles AVL son un tipo de árbol binario balanceado en altura (BA(1)) [6, 146; FOD - CLASE 6, 17]. Su construcción se determina respetando un precepto fundamental: la diferencia entre las alturas de cualquiera de sus dos subárboles con raíz común no puede superar un valor determinado, que para los árboles AVL es uno [6, 147; FOD - CLASE 6, 17]. Esto significa que, aunque la inserción y eliminación de elementos pueden ser más costosas computacionalmente para mantener el balanceo, el AVL garantiza que el árbol siempre estará balanceado.
- Eficiencia de búsqueda en AVL: Al mantenerse balanceado, el árbol AVL asegura que la altura del árbol (H) se mantiene mínima para un número dado de elementos (N), lo cual es crucial para la eficiencia. La eficiencia de búsqueda en un árbol AVL se describe específicamente como aproximadamente $1.44 * \log_2(N+2)$ [6, 147; FOD - CLASE 6, 17]. Esta fórmula confirma su naturaleza logarítmica, aunque con un factor ligeramente superior al de un árbol binario perfectamente balanceado, pero significativamente mejor que una búsqueda lineal.

-
18. Dado un árbol b de orden 100
- a. el nodo raíz puede tener solamente 3 hijos en algún momento de su construcción
 - b. el proceso de borrar un elemento del nodo raíz, puede producir un underflow en un nodo terminal
 - c. puede ser que un nodo terminal con 50 elementos, tenga 51 hijos
 - d. todas las respuestas anteriores son correctas
 - e. las opciones a y b son correctas**
 - f. las opciones b y c son correctas
 - g. las opciones a y c son correctas
 - h. ninguna opción anterior es correcta

Un árbol B de orden 100 es un tipo de árbol AVL que puede tener como máximo, por cada nodo, 99 elementos y 100 hijos [LIBRO SOLO FOD, pág. 110].

La opción **a** es correcta porque el nodo raíz es efectivamente capaz, en algún momento, de tener solamente 3 hijos. No se debe confundir con la cantidad mínima de elementos, que debe ser de $\lceil \text{Orden} / 2 \rceil - 1 = \lceil 100 / 2 \rceil - 1 = 49$ elementos como mínimo en cada nodo [LIBRO SOLO FOD, pág. 110].

La opción **b** también es correcta. Si se tiene que borrar una clave del nodo raíz, al reemplazarla con la menor clave de su subárbol derecho (por convención), puede producirse underflow si el nodo en el que se encuentra esta clave posee el número mínimo de claves (49).

La opción **c** no es correcta. Los nodos terminales, por definición, no tienen hijos.

19. Un árbol b^*

a. todos los nodos menos la raíz seguro están llenos a $2/3$ de su capacidad en todo momento

b. los nodos terminales pueden, en situaciones especiales, tener menos de $2/3$ de su capacidad ocupada

c. el nodo adyacente hermano de uno que entra en overflow siempre se puede usar para redistribuir, si no estuviera completo

d. todas las anteriores son correctas

e. las opciones a y b son correctas

f. las opciones b y c son correctas

g. las opciones a y c son correctas

h. ninguna opción es correcta

Los nodos terminales tienen, como mínimo, $\lceil (2M-1)/3 \rceil - 1$ elementos [LIBRO SOLO FOD, pág. 110].

En cambio, los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil (2M-1) / 3 \rceil$ descendientes. Por esta última razón la afirmación **a** es incorrecta (está afirmando que los nodos terminales seguro están llenos a $2/3$ de su capacidad).

Y para la afirmación **c**, no hace falta explicar demasiado ya que es un caso similar al de los árboles B y B+ convencionales. Supongamos que tenemos política de derecha para resolver underflows: si nuestro nodo entra en underflow y deseamos redistribuir con el derecho (obligatoriamente por la política), puede suceder que nuestro nodo derecho tenga el número mínimo de elementos. Entonces, aunque disponga de un nodo adyacente izquierdo “candidato” para redistribuir, mi política me dicta fusionar o concatenar. Por lo tanto no, no siempre se puede usar el nodo adyacente hermano de uno que entra en overflow para redistribuir si no estuviera completo.

20. Un árbol B es más eficiente que un árbol B+

a. porque tiene un algoritmo de inserción más eficiente

b. porque tiene un algoritmo de borrado más eficiente

c. porque tiene un algoritmo de búsqueda más eficiente

d. no, un árbol B no es más eficiente que un árbol B+

e. por todas las anteriores

f. por algunas de las anteriores

g. por ninguna de las anteriores

A chequear de todas formas, no me termina de cerrar cuál es “la más correcta”.

Un árbol B no es más eficiente que un árbol B+ en general. Ninguna de las afirmaciones que sugieren una mayor eficiencia del árbol B sobre el árbol B+ es precisa.

- **Eficiencia de Búsqueda:** La eficiencia de búsqueda en un árbol B es similar a la de los árboles B+. El tiempo de búsqueda es igual para ambos árboles [Clase 6 parte 1, pág. 47].

Esto significa que, en cuanto a la búsqueda de un registro individual, su rendimiento es comparable.

- **Eficiencia de Inserción y Eliminación:** Las operaciones de inserción y eliminación en los árboles B+ pueden requerir más tiempo en comparación con los árboles B. Esto se debe a que en los árboles B+, todos los registros se insertan y borran en los nodos terminales, y se manejan copias de elementos como separadores en nodos no terminales [Clase 6 parte 1, pág. 47].
 - **Acceso Secuencial:** La principal ventaja del árbol B+ es que "optimizan la búsqueda de información, ya que otorgan además un acceso secuencial eficiente". Esto se logra porque todos los registros de datos aparecen almacenados en los nodos terminales, los cuales están enlazados entre sí. En contraste, para los árboles B, el procesamiento secuencial es "Lento (complejo)" [Clase 6 parte 1, pág. 47].
-